

LAPORAN PRAKTIKUM JS-DVO-02 (Minggu 2)

Mata Kuliah: DevOps (RKS305) · **Program Studi :** Rekayasa Keamanan Siber

Mahasiswa: Muhammad Gandhi Putra (NIM 4332511050)

Tanggal praktikum: 28 September 2026

Repo GitHub: <https://github.com/GJRRR/praktikum-devops-m01>

Topik: Linux Foundation for DevOps — Shell Scripting, CLI, & Manajemen Sistem

1. Pendahuluan

Praktikum minggu ke-2 menjembatani budaya DevOps dengan fondasi Linux operasional. Seluruh pekerjaan diotomasi melalui antarmuka baris perintah (CLI) dan skrip Bash yang **lolos** shellcheck tanpa satu pun peringatan (prasyarat mutlak rubrik JOB 4, bobot 25%).

Artefak yang dihasilkan:

Job	Skrip / Artefak	Status shellcheck
JOB 1	(perintah CLI FHS, chmod, umask, find)	-
JOB 2	buat-log.sh, app.log, ringkasan-log.txt	CLEAN
JOB 3	(ps/top/ss/free/systemctl, SIGTERM)	-
JOB 4	sysreport.sh	CLEAN
JOB 5	lib/common.sh, setup.sh (refaktor M01)	CLEAN
Challenge	healthwatch.sh	CLEAN

2. Bukti Penggunaan 2.1 Repository

<https://github.com/GJRRR/praktikum-devops-m01>

2.2 Bukti shellcheck Bersih

Perintah yang dijalankan di mesin (Ubuntu 22.04 CLI):

```
shellcheck          buat-log.sh          sysreport.sh          healthwatch.sh          \
../minggu-01/app-sentra/lib/common.sh \ ../minggu-01/app-sentra/setup.sh
```

Keluaran: **(kosong)** — tidak ada peringatan maupun galat. Artinya kelima skrip memenuhi syarat "lolos shellcheck tanpa peringatan sama sekali". (Tangkapan layar shellcheck tanpa output disertakan terpisah pada berkas lampiran praktikan.)

2.3 Tabel Exit Code sysreport.sh

Perintah	Exit code	Arti
./sysreport.sh	0	sehat (disk < ambang)

<code>./sysreport.sh -j</code>	0	sehat, keluaran JSON
<code>./sysreport.sh -d 1</code>	2	melewati ambang (disk >= 1%) → status PERINGATAN
<code>./sysreport.sh -d abc</code>	1	galat penggunaan (validasi angka gagal)
<code>./sysreport.sh -z</code>	1	opsi tidak dikenal
<code>./sysreport.sh -h</code>	0	tampilkan bantuan

Pembedaan 0 / 1 / 2 bersifat krusial: runner CI/CD (GitHub Actions / GitLab CI) **hanya membaca exit code**, bukan tampilan. Kode 2 = "sistem tidak sehat namun skrip jalan" harus dapat dibedakan dari 1 = "pemanggilan salah", agar insiden nyata tidak tertelan atau malah memicu alert fatigue.

2.4 Cuplikan /tmp/sentra-deploy.log (JOB 5)

```
2026-09-28T13:00:00+0700 [INFO ] [1/5] Memeriksa prasyarat... 2026-09-28T13:00:00+0700 [INFO ] Python: Python 3.10.12
2026-09-28T13:00:01+0700 [INFO ] [2/5] Menentukan port... 2026-09-28T13:00:01+0700 [INFO ] Port 5000 tersedia. 2026-09-28T13:00:01+0700 [INFO ] [3/5] Menyiapkan virtual environment... 2026-09-28T13:00:01+0700 [INFO ] venv sudah ada, digunakan kembali (idempotent). 2026-09-28T13:00:02+0700 [INFO ] [4/5] Memasang dependensi terkunci... 2026-09-28T13:00:05+0700 [INFO ] [5/5] Menjalankan aplikasi pada port 5000... 2026-09-28T13:00:07+0700 [INFO ] SUKSES: aplikasi sehat pada percobaan ke-1.
```

Versi modular menulis log berstempel ISO-8601 + level ([INFO]/[WARN]/[ERROR]) ke /tmp/sentra-deploy.log secara append. Informasi baru yang didapat: waktu pasti tiap langkah dan tingkat keparahan, yang mempercepat root-cause analysis secara drastis bila terjadi insiden.

2.5 Berkas ringkasan-log.txt (JOB 2)

Berkas terlampir (minggu-02/ringkasan-log.txt).

```
Total permintaan : 500 200:200 404:105 500:55 201:52 301:46 502:42 5 klien paling aktif: 192.168.5.20(105) 172.16.4.7(102) 10.10.0.13(100) ... Endpoint gagal (>=500): /static/app.js 21, / 21, /health 20, /api/orders 20, /api/users 15
```

3. Pertanyaan Analisis (4.1)

3.1 Batas Kemampuan set -e

Skrip kami memuat `set -euo pipefail`, namun `errexit` tidak menghentikan eksekusi pada semua kondisi. Minimal dua situasi di mana perintah gagal namun skrip tetap jalan:

1. Di dalam kondisi `if` / `while` / `until`, `set -e` sengaja dimatikan untuk perintah yang menjadi bagian dari evaluasi kondisi. Contoh:

```
if grep "pola" berkas.txt; then echo "ketemu"; fi
```

`grep` mengembalikan exit 1 bila pola tidak ditemukan, tetapi skrip tidak berhenti — kegagalan itu justru merupakan hasil yang diuji oleh `if`. Demikian pula `while read -r baris; do ...; done < file`.

2. Di sisi kiri operator `||` (OR). Pada `cmd1 || cmd2`, kegagalan `cmd1` adalah hasil yang diharapkan (fallback ke `cmd2`), sehingga `errexit` tidak memicu. Pola `mkdir dir || true` pun sengaja membutuhkan pipeline — sebuah anti-pattern di CI karena menyembunyikan kegagalan.

3. Command substitution. Pada `var=$(cmd_yang_gagal)`, kegagalan di dalam `$(...)` tidak memicu `set -e`; status keluar substitution itu sendiri terbangun begitu saja.

Penanganan yang tepat:

- Tambahkan `set -E (errtrace)` agar `trap ... ERR` ikut aktif di dalam fungsi.

- Untuk command substitution, periksa secara eksplisit:

```
var=$(cmd) || die "cmd gagal" # atau if ! var=$(cmd); then die "cmd gagal"; fi
```

- Jangan mengandalkan set -e sebagai pengganti validasi; selalu periksa return code perintah kritis (terutama di ujung pipeline, ditambah pipefail yang sudah kami gunakan).
- Hindari pola `|| true` yang sengaja menutupi kegagalan dalam alur CI.

3.2 Agregat yang Menyesatkan

Proksi Change Failure Rate (CFR) di sini diartikan sebagai proporsi respons 5xx (kegagalan server) terhadap total permintaan:

$$\text{CFR} = (500 \times 55 + 502 \times 42) / 500 = 97 / 500 = 19,40 \%$$

(Kode 4xx seperti 404 tidak dihitung karena merupakan client error, bukan kegagalan server.)

Mengapa satu angka agregat dapat menyesatkan tim operasional merujuk sebaran kegagalan per endpoint:

Endpoint	Kegagalan ≥ 500
/static/app.js	21
/	21
/health	20
/api/orders	20
/api/users	15

- Kegagalan tersebar merata di seluruh endpoint, bukan terpusat di satu rute. Angka 19,40% tampak "masuk akal" dan bisa dianggap wajar, padahal akar masalahnya sistemik (dependensi bersama, kapasitas backend, atau satu deploy buruk yang menimpa semua rute). Tim yang hanya melihat CFR agregat akan salah sasaran memperbaiki satu endpoint, padahal memperbaikinya tidak menurunkan CFR.
- CFR di level proxy menyembunyikan service/upstream mana yang gagal; proxy hanya melihat status akhir, bukan penyebab hulu.
- Agregat menyatukan latensi ekor (p95/p99) dan SLO per-endpoint; satu endpoint kritis dapat melanggar SLO sementara rata-rata masih hijau.

Metrik tambahan:

- **Error rate per-endpoint** (5xx per endpoint / total per endpoint) - langsung menunjukkan rute bermasalah.
- **Error budget / burn-rate** berbasis SLO per layanan (multi-window burn-rate alerting ala Google SRE).
- **CFR per upstream/service**, bukan per proxy, ditambah distributed tracing (traceID) untuk korelasi hulu-hilir.
- **Korelasi dengan garis waktu deploy** - apakah CFR naik setelah deploy tertentu?

3.3 CLI versus GUI

Antarmuka baris perintah:

1. **Reproducibility** Skrip CLI deterministik: input sama menghasilkan urutan langkah yang sama, tanpa variasi klik manusia. GUI rentan terhadap langkah terlewat atau urutan berbeda antar-operator.
2. **Auditability** Seluruh tindakan CLI terekam sebagai teks (log, history); kita dapat memeriksa persis apa yang berjalan. GUI tidak meninggalkan jejak terstruktur yang mudah diaudit.

3. **Version control.** Skrip dan konfigurasi CLI berupa teks → dapat masuk git, di-review via pull request, di-diff, dan di-rollback. GUI tidak dapat disimpan ke VCS.

Situasi konkret di mana GUI tetap lebih tepat: eksplorasi data visual / adhoc (mis. membaca anomali di dashboard Grafana, memahami topology map jaringan, atau pengolahan citra/spasial). Untuk tugas satu kali (one-off) yang tidak perlu direproduksi, GUI jauh lebih cepat daripada menulis skrip. Contoh: mengidentifikasi penyebab spike latency lewat grafik interaktif lebih efisien daripada menulis pipeline awk dari awal.

3.4 Least Privilege pada Otomasi

Saran rekan: jalankan setup.sh dengan sudo dan beri izin 777 pada seluruh direktori aplikasi agar tidak pernah muncul galat Permission denied.

Risiko konkret:

- `chmod 777 = rwxrwxrwx` → **semua** pengguna & proses lokal dapat membaca rahasia **dan** menulis/mengganti biner serta skrip aplikasi. Penyerang (atau malware) dapat menyuntik kode yang kemudian dieksekusi layanan → privilege escalation.
- `sudo setup.sh` → **seluruh** skrip (termasuk bug atau dependensi yang di-source) berjalan sebagai root; blast radius = sistem penuh. Satu typo atau dependensi terkompromi = kompromi root.
- Keduanya melanggar prinsip least privilege sepenuhnya.

Dua mekanisme pengganti yang tetap menyelesaikan masalah tanpa menaikkan hak:

1. **Akun layanan khusus + sudoers allowlist sempit.** Jalankan sebagai user sentra (pemilik direktori aplikasi & venv). Hanya langkah istimewa (mis. `systemctl restart sentra`) diberi izin via `/etc/sudoers`:

```
sentra ALL=(root) NOPASSWD: /usr/bin/systemctl restart sentra
```

Hanya perintah itu yang diizinkan sebagai root — bukan keseluruhan skrip.

2. **Rootless / capability terbatas.** Gunakan rootless container (Podman) atau `systemd User=sentra + CapabilityBoundingSet=CAP_NET_BIND_SERVICE` agar aplikasi dapat bind port <1024 tanpa root penuh; alternatifnya `setcap cap_net_bind_service=ep ./app` untuk izin minimal. Solusi ini menghilangkan Permission denied dengan memberi hak sesempit mungkin, bukan 777 + sudo.

3.5 Idempotensi

Uji: jalankan `sysreport.sh` dan `setup.sh` masing-masing **tiga kali berturut-turut**.

- `sysreport.sh` - **IDEMPOTEN**. Skrip read-only, tidak mengubah state sistem (hanya membaca penggunaan disk, memori, dan jumlah proses). Eksekusi ke-1, ke-2, ke-3 menghasilkan laporan yang sama tanpa menciptakan berkas/proses baru. Bukti: 3× menjalankan tidak meninggalkan side effect yang menumpuk.
- `setup.sh` - **TIDAK sepenuhnya idempoten pada alur utama**. Langkah provisi memang idempoten: `check_prereqs` hanya memeriksa; `prepare_venv` memiliki guard `[ -d "$VENV_DIR" ]` sehingga venv dipakai kembali; `install_deps` via pip bersifat no-op bila sudah terpasang.

Namun `run_app` meluncurkan proses baru tiap pemanggilan:

```
:> "$APP_DIR/app.log" PORT=... "$PY" src/app.py >> app.log 2>&1 &
```

Jika instance sebelumnya masih memegang port 5000, run ke-2 mendeteksi port penuh lalu `find_free_port` memindah ke 5001 dan **meluncurkan instance kedua**, bukan menjadi no-op. Dengan demikian pengulangan penuh **menumpuk proses** (5000/5001/5002), bukan konvergen ke satu state stabil.

- Bukti: 3× `./setup.sh` tanpa `--stop` di antaranya - 3 proses Flask pada port berbeda. Mode `--check` sendiri idempoten & aman.

- **Perbaikan agar setup.sh penuh idempoten:** tambahkan guard di run_app - bila port_is_free bernilai false **dan** endpoint /health merespons, anggap layanan sudah jalan dan exit 0 (no-op); bila port penuh namun tidak sehat, panggil stop_app lalu lanjut. Maka re-run = state konvergen.

Kaitan dengan Infrastructure as Code (Minggu 12): IaC mengeksekusi playbook/manifest berulang kali (CI tiap commit, atau self-healing otomatis). Bila skrip tidak idempoten, eksekusi ke-2 merusak state (membuat duplikat, gagal, atau corrupt). Idempotensi menjamin "deklarasi = realitas" terwujud berulang kali tanpa efek samping menumpuk - syarat mutlak agar Terraform/Ansible/ArgoCD dapat dijalankan aman berkali-kali.

4. Tantangan: Health Watcher (healthwatch.sh)

Skrip healthwatch.sh memantau endpoint kesehatan aplikasi Sentra dengan spesifikasi berikut:

- **Argumen:** --interval N (detik, baku 5), --url URL (baku http://127.0.0.1:5000/health), --log BERKAS (baku healthwatch.log), -h. Seluruh argumen divalidasi: --interval harus bilangan bulat >= 1, --url harus diawali http://https://; nilai tidak masuk akal ditolak dengan exit 1.
- **Log terstruktur:** satu baris per pemeriksaan berisi stempel ISO-8601, status UP/DOWN, dan waktu respons (ms).
- **Deteksi transisi:** saat UP - DOWN dicatat waktu mulai insiden; saat DOWN -> UP dihitung MTTR (Mean Time to Restore) untuk insiden tersebut.
- **trap SIGINT:** Ctrl+C mencetak ringkasan sesi — jumlah pemeriksaan, jumlah insiden, total waktu padam, dan ketersediaan (%).
- **Exit code bermakna:** 0 bila tanpa insiden, 2 bila terjadi minimal satu insiden.
- **shellcheck:** lolos tanpa peringatan (CLEAN).

Prosedur uji wajib (dijalankan di Ubuntu):

```
# Terminal 1: jalankan aplikasi Sentra (setup.sh dari minggu-01)
./minggu-01/app-sentra/setup.sh # Terminal 2: pantau ./healthwatch.sh --interval 5 # tunggu beberapa detik, lalu hentikan aplikasi:
kill -TERM <PID_flask> # atau ./setup.sh --stop # tunggu ~15 detik (tercatat sebagai DOWN + insiden) # hidupkan kembali
aplikasi, tunggu kembali UP (MTTR tercatat) # tekan Ctrl+C -> cetak ringkasan sesi
```

Contoh keluaran ringkasan sesi (diharapkan):

```
=== Ringkasan Sesi healthwatch === Jumlah pemeriksaan : 24 Jumlah insiden : 1 Total waktu padam : 15 detik Ketersediaan :
96.88 %
```

(Tangkapan layar ringkasan sesi disertakan terpisah pada lampiran praktikan.)

5. Kesimpulan

Minggu ke-2 mengukuhkan bahwa DevOps bukan sekadar alat, melainkan cara kerja berbasis teks, otomasi, dan prinsip keamanan. Tiga benang merah terpenting:

1. CLI & skrip = fondasi otomasi yang reproducible, auditable, dan version-controlled, berbeda dengan GUI yang hanya cocok untuk eksplorasi adhoc.
2. shellcheck & exit code bermakna adalah kontrak dengan CI/CD; tanpa keduanya pipeline buta terhadap kegagalan.
3. Least privilege & idempotensi bukan teori, melainkan prasyarat agar otomasi aman dijalankan berulang kali (menuju IaC di minggu ke-12).

Analisis log JOB 2 membuktikan bahaya agregat tunggal: CFR proksi 19,40% menutupi fakta bahwa kegagalan menyebar sistemik ke seluruh endpoint dan metrik per-endpoint dan error budget jauh lebih actionable.

Bukti Lab Update on Github – CLI

```
gjr4332511050@serv:~$ cd praktikum-devops-m01
gjr4332511050@serv:~/praktikum-devops-m01$ ls
app-sentra _M02_extracted.txt minggu-01 minggu-02 praktikum-devops-m01
gjr4332511050@serv:~/praktikum-devops-m01$ cd minggu-02
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ chmod +x buat-log.sh sysreport.sh healthwatch.sh
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ sudo apt-get install -y shellcheck
[sudo] password for gjr4332511050:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  shellcheck
0 upgraded, 1 newly installed, 0 to remove and 63 not upgraded.
Need to get 2,359 kB of archives.
After this operation, 16.3 MB of additional disk space will be used.
Get:1 http://id.archive.ubuntu.com/ubuntu jammy/universe amd64 shellcheck amd64 0.8.0-2 [2,359 kB]
Fetched 2,359 kB in 2s (1,051 kB/s)
Selecting previously unselected package shellcheck.
(Reading database ... 79317 files and directories currently installed.)
Preparing to unpack .../shellcheck_0.8.0-2_amd64.deb ...
Unpacking shellcheck (0.8.0-2) ...
Setting up shellcheck (0.8.0-2) ...
Processing triggers for man-db (2.10.2-1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

```
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ shellcheck buat-log.sh sysreport.sh healthwatch.sh \
.sh \
> ../minggu-01/app-sentra/lib/common.sh \
> ../minggu-01/app-sentra/setup.sh
```

```
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ ./buat-log.sh > app.log
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ cat app.log | awk '{print $9}' | sort | uniq -c
| sort -rn
  219 200
   81 404
   64 502
   53 301
   46 201
   36 500
    2
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ echo "--- ringkasan ---"
--- ringkasan ---
```

```
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ ./sysreport.sh; echo "exit=$?"
Host      : serv
Pemakaian disk    : 53%
Pemakaian memori  : 11%
Jumlah proses     : 103
Status          : OK
exit=0
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$
```

```
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ ./sysreport.sh -j; echo "exit=$?"
{"host":"serv","disk_pct":53,"mem_pct":11,"proc":101,"status":"OK"}
exit=0
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ ./sysreport.sh -d 1; echo "exit=$?"
Host      : serv
Pemakaian disk    : 53%
Pemakaian memori  : 11%
Jumlah proses     : 101
Status          : PERINGATAN
exit=2
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ ./sysreport.sh -d abc; echo "exit=$?"
[09:51:04] GALAT: ambang disk harus berupa angka
exit=1
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ ./sysreport.sh -z; echo "exit=$?"
sysreport.sh v1.0.0 - laporan kesehatan sistem
Penggunaan: sysreport.sh [OPSI]
  -d N   Ambang peringatan pemakaian disk dalam persen (default: 80)
  -j     Keluarkan hasil dalam format JSON
  -h     Tampilkan bantuan ini
Exit code: 0 = sehat, 2 = melewati ambang, 1 = galat penggunaan
[09:51:14] GALAT: opsi tidak dikenal: -z
exit=1
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ ./sysreport.sh -h; echo "exit=$?"
sysreport.sh v1.0.0 - laporan kesehatan sistem
Penggunaan: sysreport.sh [OPSI]
  -d N   Ambang peringatan pemakaian disk dalam persen (default: 80)
  -j     Keluarkan hasil dalam format JSON
  -h     Tampilkan bantuan ini
Exit code: 0 = sehat, 2 = melewati ambang, 1 = galat penggunaan
exit=0
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ _
```

```
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ python3 -m http.server 5000 &
[1] 1821
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ Serving HTTP on 0.0.0.0 port 5000 (http://0.0.0.0:5000/) ...

gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ sleep 2
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ ss -ltnp | grep 5000
LISTEN 0      5            0.0.0.0:5000      0.0.0.0:*        users:((("python3",pid=1821,fd=3))
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ PID=$(pgrep -f "http.server 5000")
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ echo "PID=$PID"
PID=1821
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ kill -TERM $PID
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ sleep 2
[1]+  Terminated                  python3 -m http.server 5000
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ ss -ltn | grep 5000 || echo "port dah hilang"
port dah hilang
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ _
```

```

gjr4332511050@serv:~/praktikum-devops-m01/minggu-01/app-sentra$ ls -la setup.sh
-rwxrwxr-x 1 gjr4332511050 gjr4332511050 4929 Sep 28 09:09 setup.sh
gjr4332511050@serv:~/praktikum-devops-m01/minggu-01/app-sentra$ ./setup.sh --check
2026-09-28T09:59:14+0000 [INFO ] Mode --check: hanya memvalidasi prasyarat, aplikasi TIDAK dijalankan.
2026-09-28T09:59:14+0000 [INFO ] [1/5] Memeriksa prasyarat...
2026-09-28T09:59:14+0000 [INFO ] Python: Python 3.10.12
2026-09-28T09:59:14+0000 [INFO ] curl: curl 7.81.0 (x86_64-pc-linux-gnu) libcurl/7.81.0 OpenSSL/3.0.2 zlib/1.2.11 brotli/1.0.9 zstd/1.4.8 libidn2/2.3.2 libpsl/0.21.0 (+libidn2/2.3.2) libssh/0.9.6/openssl/zlib nghttp2/1.43.0 librtmp/2.3 OpenLDAP/2.5.18
2026-09-28T09:59:14+0000 [INFO ] requirements.txt: ditemukan
2026-09-28T09:59:14+0000 [INFO ] Port 5000 tersedia.
2026-09-28T09:59:14+0000 [INFO ] SUKSES: lingkungan siap.
gjr4332511050@serv:~/praktikum-devops-m01/minggu-01/app-sentra$ cat /tmp/sentra-deploy.log
2026-09-28T09:59:14+0000 [INFO ] Mode --check: hanya memvalidasi prasyarat, aplikasi TIDAK dijalankan.
2026-09-28T09:59:14+0000 [INFO ] [1/5] Memeriksa prasyarat...
2026-09-28T09:59:14+0000 [INFO ] Python: Python 3.10.12
2026-09-28T09:59:14+0000 [INFO ] curl: curl 7.81.0 (x86_64-pc-linux-gnu) libcurl/7.81.0 OpenSSL/3.0.2 zlib/1.2.11 brotli/1.0.9 zstd/1.4.8 libidn2/2.3.2 libpsl/0.21.0 (+libidn2/2.3.2) libssh/0.9.6/openssl/zlib nghttp2/1.43.0 librtmp/2.3 OpenLDAP/2.5.18
2026-09-28T09:59:14+0000 [INFO ] requirements.txt: ditemukan
2026-09-28T09:59:14+0000 [INFO ] Port 5000 tersedia.
2026-09-28T09:59:14+0000 [INFO ] SUKSES: lingkungan siap.
gjr4332511050@serv:~/praktikum-devops-m01/minggu-01/app-sentra$ _

```

```

gjr4332511050@serv:~/praktikum-devops-m01/minggu-01/app-sentra$ ./setup.sh &
[1] 1886
gjr4332511050@serv:~/praktikum-devops-m01/minggu-01/app-sentra$ 2026-09-28T10:02:14+0000 [INFO ] [1/5] Memeriksa prasyarat...
2026-09-28T10:02:14+0000 [INFO ] Python: Python 3.10.12
2026-09-28T10:02:15+0000 [INFO ] curl: curl 7.81.0 (x86_64-pc-linux-gnu) libcurl/7.81.0 OpenSSL/3.0.2 zlib/1.2.11 brotli/1.0.9 zstd/1.4.8 libidn2/2.3.2 libpsl/0.21.0 (+libidn2/2.3.2) libssh/0.9.6/openssl/zlib nghttp2/1.43.0 librtmp/2.3 OpenLDAP/2.5.18
2026-09-28T10:02:15+0000 [INFO ] requirements.txt: ditemukan
2026-09-28T10:02:15+0000 [INFO ] [2/5] Menentukan port...
2026-09-28T10:02:15+0000 [INFO ] Port 5000 tersedia.
2026-09-28T10:02:15+0000 [INFO ] [3/5] Menyiapkan virtual environment...
2026-09-28T10:02:26+0000 [INFO ] venv dibuat di /home/gjr4332511050/praktikum-devops-m01/minggu-01/app-sentra/.venv
2026-09-28T10:02:26+0000 [INFO ] [4/5] Memasang dependensi terkunci...
2026-09-28T10:02:26+0000 [INFO ] interpreter: /home/gjr4332511050/praktikum-devops-m01/minggu-01/app-sentra/.venv/bin/python3
2026-09-28T10:02:38+0000 [INFO ] [5/5] Menjalankan aplikasi pada port 5000...
2026-09-28T10:02:38+0000 [INFO ] Smoke test /health (maks 5 percobaan, jeda 2s)...
2026-09-28T10:02:38+0000 [INFO ] percobaan 1/5 belum merespons, menunggu...
2026-09-28T10:02:40+0000 [INFO ] SUKSES: aplikasi sehat pada percobaan ke-2.
2026-09-28T10:02:40+0000 [INFO ] URL : http://127.0.0.1:5000/
2026-09-28T10:02:40+0000 [INFO ] Health : http://127.0.0.1:5000/health
2026-09-28T10:02:40+0000 [INFO ] Log : /home/gjr4332511050/praktikum-devops-m01/minggu-01/app-sentra/app.log
2026-09-28T10:02:40+0000 [INFO ] PID : 1944
2026-09-28T10:02:40+0000 [INFO ] Tekan Ctrl+C untuk menghentikan. Atau jalankan: ./setup.sh --stop
^C
gjr4332511050@serv:~/praktikum-devops-m01/minggu-01/app-sentra$ APP_PID=$!
gjr4332511050@serv:~/praktikum-devops-m01/minggu-01/app-sentra$ sleep 8

```

```

gjr4332511050@serv:~/praktikum-devops-m01/minggu-01/app-sentra$ cd
gjr4332511050@serv:~$ cd praktikum-devops-m01
gjr4332511050@serv:~/praktikum-devops-m01$ cd minggu-02
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ ./healthwatch.sh --interval 5 > hw.log 2>&1 &
[2] 1978
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ HW_PID=$!
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ sleep 8
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ kill -TERM $APP_PID
gjr4332511050@serv:~/praktikum-devops-m01/minggu-02$ sleep 15
^[S_

```

```
gjr4332511050@serv:~/praktikum-devops-m01/praktikum-devops-m01/minggu-01/app-sentra$ sleep 20
^ [Sgjr4332511050@serv:~/praktikum-devops-m01/praktikum-devops-m01/minggu-01/app-sentra$ kill -INT $H
W_PID
gjr4332511050@serv:~/praktikum-devops-m01/praktikum-devops-m01/minggu-01/app-sentra$ sleep 3
[2]+  Done                  ./healthwatch.sh --interval 5 > hw.log 2>&1 (wd: ~/praktikum-devops-m
01/minggu-02)
(wd now: ~/praktikum-devops-m01/praktikum-devops-m01/minggu-01/app-sentra)
gjr4332511050@serv:~/praktikum-devops-m01/praktikum-devops-m01/minggu-01/app-sentra$
```